

TUGAS TERSTRUKTUR

OPTIMASI RUTE PENGIRIMAN MULTI-KRITERIA MENGUNAKAN ALGORITMA A* DENGAN HEURISTIK REVERSE UNIFORM COST SEARCH



Disusun Oleh :

Dennita Noor Febianty	(26/589354/PPA/07353)
Mar'iy Romizzidi Amly	(26/589558/PPA/07365)
Fatimah Azzahro	(26/590318/PPA/07393)

KELOMPOK 5

**PROGRAM STUDI MAGISTER KECERDASAN ARTIFISIAL
DEPARTEMEN ILMU KOMPUTER DAN ELEKTRONIKA
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS GADJAH MADA
YOGYAKARTA
2026**

Optimasi Rute Pengiriman Multi-Kriteria Menggunakan Algoritma A* dengan Heuristik Reverse Uniform Cost Search

Dennita Noor Febianty, Mar'iy Romizzidi Amly, and Fatimah Azzahro
Kelompok 5

Program Studi Magister Kecerdasan Artifisial
Departemen Ilmu Komputer dan Elektronika
Fakultas Matematika dan Ilmu Pengetahuan Alam
Universitas Gadjah Mada, Yogyakarta

Abstract—Optimasi rute pengiriman pada jaringan jalan dapat dirumuskan sebagai masalah pencarian pada graf berbobot dengan lebih dari satu pertimbangan biaya. Pembahasan ini menggunakan A* sebagai algoritma pencarian dan Reverse Uniform Cost Search (Reverse UCS) sebagai pembentuk nilai heuristik atau cost-to-go. Fungsi biaya menggabungkan jarak efektif dan waktu tempuh, sedangkan biaya bahan bakar dimasukkan sebagai penalti pada jarak efektif. Untuk membuat kedua komponen berada pada skala yang sebanding, jarak efektif dan waktu dinormalisasi terhadap nilai maksimum masing-masing. Reverse UCS kemudian menghitung batas bawah biaya menuju tujuan secara terpisah untuk komponen jarak dan waktu, lalu menggabungkannya sesuai bobot layanan. Data jaringan terdiri atas 55 node, 103 koneksi jalan unik, empat kendaraan, tiga jenis bahan bakar, dan empat paket layanan. Sebanyak 16 skenario rute dianalisis untuk memperlihatkan pengaruh bobot layanan terhadap jarak, waktu, dan biaya. Pembahasan difokuskan pada dasar matematis, alasan pemilihan formulasi, mekanisme heuristik, pseudocode, serta interpretasi hasil.

Index Terms—A*, Reverse Uniform Cost Search, route optimization, weighted graph, fuel cost, heuristic

I. PENDAHULUAN

Optimasi rute merupakan salah satu bentuk masalah pencarian yang dapat direpresentasikan menggunakan graf. Dalam konteks pengiriman, pemilihan rute tidak selalu cukup dilakukan berdasarkan jarak terpendek karena rute yang lebih pendek belum tentu mempunyai waktu tempuh yang lebih rendah. Di sisi lain, penggunaan kendaraan dan bahan bakar yang berbeda memberikan konsekuensi biaya yang berbeda pula. Oleh karena itu, diperlukan suatu fungsi objektif yang dapat menggabungkan beberapa komponen tersebut ke dalam satu ukuran biaya.

Pendekatan yang digunakan dalam tugas ini adalah A*. A* memilih node berdasarkan gabungan biaya yang telah ditempuh dan perkiraan biaya yang masih diperlukan untuk mencapai tujuan. Agar perkiraan menuju tujuan tidak sekadar menggunakan ukuran geometris, Reverse UCS digunakan untuk memperoleh cost-to-go berdasarkan jaringan jalan yang sebenarnya. Dengan demikian, heuristic dibentuk dari struktur jaringan dan fungsi biaya yang sama dengan fungsi objektif pencarian.

II. PENJELASAN MASALAH

Jaringan jalan dinyatakan sebagai graf berbobot

$$G = (V, E), \quad (1)$$

dengan V sebagai himpunan node dan E sebagai himpunan koneksi jalan. Setiap koneksi memiliki jarak d_e dalam kilometer dan waktu tempuh $T(e)$ dalam menit. Waktu tempuh diperlakukan sebagai atribut yang telah tersedia pada setiap koneksi, sehingga tidak diturunkan dari kecepatan kendaraan.

Selain karakteristik jalan, setiap skenario memiliki kendaraan k , jenis bahan bakar b , dan layanan dengan bobot jarak J_W serta bobot waktu W_W . Tujuan pencarian adalah memperoleh jalur P dari node awal s menuju node tujuan t yang meminimalkan fungsi biaya gabungan:

$$C(P) = J_W \frac{D_{eff}(P)}{D_{scale}} + W_W \frac{T(P)}{T_{scale}}. \quad (2)$$

Dalam implementasi, bentuk fungsi objektif tersebut direalisasikan melalui perhitungan $g(n)$ pada fungsi berikut.

Persamaan (2) menjadi dasar seluruh pembentukan biaya. Implementasi lengkap pembentukan biaya kumulatif ditunjukkan pada potongan kode `calculate_g_score` pada bagian berikutnya. Dengan demikian, setiap komponen pada tahap berikutnya memiliki hubungan yang jelas: jarak dan efisiensi menentukan konsumsi bahan bakar, konsumsi menentukan biaya bahan bakar, biaya tersebut memengaruhi jarak efektif, kemudian jarak efektif dan waktu dinormalisasi sebelum digabungkan menjadi biaya pencarian.

III. INFORMASI DATA

A. Jaringan Jalan

Jaringan terdiri atas 55 node, yaitu 26 titik pengantaran A–Z dan 29 titik persimpangan AA–BC. Terdapat 103 koneksi jalan unik yang bersifat dua arah. Karena setiap koneksi dua arah dapat ditelusuri dari kedua ujungnya, representasi adjacency memiliki 206 entri.

Setiap koneksi menyimpan dua atribut numerik, yaitu jarak dan waktu tempuh. Jarak digunakan dalam perhitungan konsumsi bahan bakar dan jarak efektif, sedangkan waktu

digunakan langsung sebagai komponen biaya waktu. Tidak digunakan parameter kecepatan maksimum kendaraan untuk menentukan waktu tempuh.

Nilai jarak dan waktu untuk satu koneksi diambil secara langsung melalui fungsi berikut.

Listing 1. Potongan kode pengambilan jarak dan waktu tempuh edge.

```
def get_distance(current, neighbor):
    """Mengambil jarak aktual antar-node."""
    try:
        return DISTANCE[current][neighbor]
    except KeyError:
        raise ValueError(
            f"Distance untuk edge {current} -> {neighbor}
            ↪ tidak ditemukan."
        )

def get_travel_time(current, neighbor):
    """Mengambil waktu tempuh aktual antar-node."""
    try:
        return GRAPH[current][neighbor]
    except KeyError:
        raise ValueError(
            f"Travel time untuk edge {current} -> {neighbor}
            ↪ tidak ditemukan."
        )
```

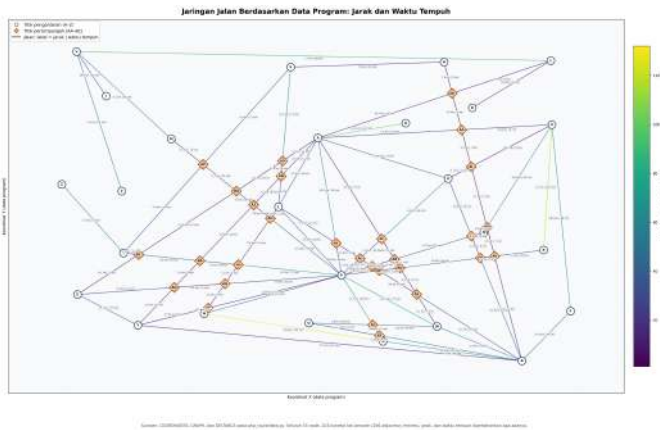


Fig. 1. Visualisasi jaringan jalan berdasarkan data program pada pka_route/data.py. Setiap label edge menampilkan jarak (km) dan waktu tempuh statis (menit). Lingkaran menunjukkan titik pengantaran A–Z, sedangkan berlian menunjukkan titik persimpangan AA–BC. Posisi node menggunakan koordinat yang tersimpan pada program.

B. Kendaraan

Efisiensi kendaraan dinyatakan dalam kilometer per liter. Nilai tersebut hanya digunakan untuk menentukan jumlah bahan bakar yang diperlukan pada jarak tertentu.

TABLE I
KARAKTERISTIK KENDARAAN

ID	Kendaraan	Efisiensi (km/L)
V1	Honda Scoopy	59.0
V2	Honda Beat	60.6
V3	Honda Vario 160	46.9
V4	Yamaha NMAX	40.0

Nilai tersebut dinyatakan secara eksplisit pada data kendaraan sebagai berikut.

Listing 2. Potongan kode data kendaraan.

```
VEHICLE_DATABASE = {
    "1": ("Honda Scoopy", 59.0), "2": ("Honda Beat", 60.6),
    "3": ("Honda Vario 160", 46.9), "4": ("Yamaha NMAX",
    ↪ 40.0)
}
```

C. Bahan Bakar

Harga bahan bakar merupakan parameter masukan yang digunakan untuk menghitung biaya bahan bakar per satuan jarak.

TABLE II
KARAKTERISTIK BAHAN BAKAR

ID	Bahan bakar	RON	Harga/L (Rp)
B1	Pertalite	90	10,000
B2	Pertamax	92	13,500
B3	Pertamax Turbo	98	14,750

Harga bahan bakar dinyatakan secara eksplisit pada data bahan bakar sebagai berikut.

Listing 3. Potongan kode data bahan bakar.

```
FUEL_DATABASE = {
    "1": ("Pertalite (RON 90)", 10000), "2": ("Pertamax (RON
    ↪ 92)", 13500),
    "3": ("Pertamax Turbo (RON 98)", 14750)
}
```

D. Paket Layanan

Bobot layanan menentukan tingkat kepentingan relatif antara jarak dan waktu. Seluruh pasangan bobot memenuhi $J_W + W_W = 1$, sehingga fungsi objektif merupakan kombinasi berbobot dari dua komponen yang telah dinormalisasi.

TABLE III
BOBOT PAKET LAYANAN

ID	Layanan	J_W	W_W
S1	Reguler (Seimbang)	0.50	0.50
S2	Kilat (Prioritas Waktu)	0.20	0.80
S3	Express (Waktu Agresif)	0.10	0.90
S4	Hemat (Prioritas Jarak)	0.80	0.20

Bobot tersebut digunakan kembali pada pencarian melalui dictionary layanan.

Listing 4. Potongan kode definisi bobot layanan.

```
SERVICE_DATABASE = {
    "1": ("Reguler (Seimbang)", {"J_W": 0.50, "W_W": 0.50}),
    "2": ("Kilat (Prioritas Waktu)", {"J_W": 0.20, "W_W":
    ↪ 0.80}),
    "3": ("Express (Waktu Agresif)", {"J_W": 0.10, "W_W":
    ↪ 0.90}),
}
```

```
"4": ("Hemat (Prioritas Jarak)", {"J_W": 0.80, "W_W":
  ↪ 0.20})
}
```

IV. METODE DAN DASAR MATEMATIS

A. Konsumsi Bahan Bakar

Misalkan sebuah kendaraan k memiliki efisiensi E_k km/L dan menempuh satu edge dengan jarak d_e km. Banyaknya bahan bakar yang diperlukan adalah

$$F(e, k) = \frac{d_e}{E_k}. \quad (3)$$

Implementasi tidak menyimpan variabel $F(e, k)$ secara terpisah. Hubungan yang sama direalisasikan melalui pembentukan harga bahan bakar per kilometer, yaitu harga per liter dibagi efisiensi kendaraan.

Listing 5. Potongan kode yang merepresentasikan hubungan jarak, efisiensi, dan biaya bahan bakar.

```
fuel_price_km = (
    fuel_type[1] / vehicle[1]
)
```

Persamaan (3) digunakan sebagai bentuk matematis untuk menjelaskan hubungan tersebut, sedangkan implementasi langsung menggunakan rasio P_b/E_k . Jika kendaraan mampu menempuh E_k kilometer dengan satu liter, maka setiap kilometer membutuhkan $1/E_k$ liter. Untuk jarak d_e , kebutuhan bahan bakar menjadi d_e/E_k liter. Efisiensi yang lebih tinggi menghasilkan konsumsi yang lebih rendah untuk jarak yang sama.

B. Biaya Bahan Bakar

Jika harga bahan bakar adalah P_b rupiah per liter, biaya untuk menempuh edge tersebut adalah

$$B(e, k, b) = F(e, k)P_b = \frac{d_e}{E_k}P_b. \quad (4)$$

Pada implementasi, biaya bahan bakar dihitung dengan fungsi berikut.

Listing 6. Potongan kode perhitungan biaya bahan bakar per edge.

```
def calculate_fuel_cost(distance, fuel_price_km):
    return distance * fuel_price_km
```

Persamaan ini dapat ditulis menggunakan harga bahan bakar per kilometer:

$$P_{km} = \frac{P_b}{E_k}, \quad (5)$$

Pembentukan P_{km} menggunakan potongan kode yang sama seperti pada pembahasan konsumsi bahan bakar sebelumnya, yaitu `fuel_price_km = fuel_type[1] / vehicle[1]`.

sehingga

$$B(e, k, b) = d_e P_{km}. \quad (6)$$

Penggunaan P_{km} untuk menghitung biaya bahan bakar ditunjukkan langsung pada fungsi `edge_metrics` di bawah.

Bentuk (6) memperlihatkan bahwa biaya bahan bakar meningkat secara linear terhadap jarak. Untuk kendaraan yang sama, bahan bakar dengan harga per liter lebih tinggi menghasilkan biaya yang lebih besar. Untuk bahan bakar yang sama, kendaraan dengan efisiensi lebih rendah menghasilkan biaya per kilometer yang lebih tinggi.

Pengambilan ketiga metrik edge dilakukan melalui fungsi berikut.

Listing 7. Potongan kode penggabungan jarak, waktu, dan biaya bahan bakar pada edge.

```
def edge_metrics(current, neighbor, fuel_price_km):
    distance = get_distance(current, neighbor)
    travel_time = get_travel_time(current, neighbor)

    fuel_cost = calculate_fuel_cost(
        distance,
        fuel_price_km
    )

    return distance, travel_time, fuel_cost
```

C. Jarak Efektif

Jarak fisik dan biaya bahan bakar mempunyai satuan yang berbeda, sehingga keduanya tidak dapat dijumlahkan secara langsung. Pendekatan yang digunakan adalah mengubah biaya bahan bakar menjadi penalti dalam satuan jarak melalui konstanta skala 1000:

$$D_{eff}(e) = d_e + \frac{B(e, k, b)}{1000}. \quad (7)$$

Persamaan tersebut direalisasikan secara langsung pada fungsi berikut.

Listing 8. Potongan kode perhitungan jarak efektif.

```
def calculate_effective_distance(distance, fuel_cost):
    return distance + (fuel_cost / 1000.0)
```

Konstanta 1000 pada persamaan tersebut merupakan konstanta skala, bukan faktor fisik seperti kecepatan, kapasitas mesin, atau konsumsi tambahan kendaraan. Fungsinya adalah mengubah besarnya biaya rupiah menjadi penalti jarak yang dapat digabungkan dengan d_e . Dengan demikian, semakin besar biaya bahan bakar suatu edge, semakin besar pula jarak efektifnya.

Secara konseptual, persamaan ini menjadikan biaya bahan bakar sebagai penalti ekonomi yang memengaruhi komponen jarak. Konsekuensinya, bahan bakar dan kendaraan tidak memperoleh bobot ketiga yang terpisah; pengaruh keduanya terlebih dahulu diserap ke dalam D_{eff} . Hal ini membuat fungsi objektif tetap memiliki dua dimensi utama, yaitu jarak efektif dan waktu.

a) *Contoh perhitungan.*: Pada edge B-AE, jaraknya 9.7 km. Untuk Honda Scoopy dengan efisiensi 59.0 km/L dan harga Pertalite Rp10,000/L,

$$F = \frac{9.7}{59.0} = 0.1644 \text{ L}, \quad (8)$$

$$B = 0.1644 \times 10,000 = \text{Rp } 1,644.07, \quad (9)$$

sehingga

$$D_{eff} = 9.7 + \frac{1,644.07}{1000} = 11.3441 \text{ km-ekuivalen}. \quad (10)$$

Contoh tersebut menunjukkan bahwa jarak efektif lebih besar daripada jarak fisik karena terdapat penalti biaya bahan bakar.

D. Normalisasi dengan Nilai Maksimum

Jarak efektif dan waktu mempunyai skala yang berbeda. Tanpa normalisasi, komponen yang mempunyai angka lebih besar dapat mendominasi penjumlahan meskipun bobot layanan dibuat sama. Karena itu, kedua komponen dinormalisasi.

Kedua komponen dinormalisasi terhadap nilai maksimumnya:

$$D_{norm}(e) = \frac{D_{eff}(e)}{D_{scale}}, \quad T_{norm}(e) = \frac{T(e)}{T_{scale}}. \quad (11)$$

Implementasi normalisasi menggunakan pembagi yang telah dihitung untuk konfigurasi kendaraan–bahan bakar.

Listing 9. Potongan kode normalisasi jarak efektif dan waktu.

```
normalized_distance = (
    effective_distance /
    norm["D_SCALE"]
)

normalized_time = (
    time /
    norm["T_SCALE"]
)
```

Nilai pembagi ditentukan sebagai

$$D_{scale} = \max_{e \in E} D_{eff}(e), \quad T_{scale} = \max_{e \in E} T(e). \quad (12)$$

Konstanta tersebut dibentuk untuk seluruh edge pada jaringan.

Listing 10. Potongan kode pembentukan konstanta normalisasi.

```
max_effective_distance = 0.0
max_time = 0.0

for node, neighbors in GRAPH.items():
    for neighbor in neighbors:
        distance = get_distance(
            node,
            neighbor
        )

        travel_time = get_travel_time(
            node,
            neighbor
        )

        fuel_cost = calculate_fuel_cost(
            distance,
            fuel_price_km
        )

        effective_distance = calculate_effective_distance(
            distance,
            fuel_cost
        )
```

```
max_effective_distance = max(
    max_effective_distance,
    effective_distance
)

max_time = max(
    max_time,
    travel_time
)

return {
    "D_SCALE": max(max_effective_distance, 1.0),
    "T_SCALE": max(max_time, 1.0),
}
```

Selain nilai maksimum, implementasi memberikan batas bawah 1.0 pada kedua konstanta normalisasi. Jadi bentuk matematis menggunakan nilai maksimum jaringan, sedangkan kode menambahkan pengaman agar penyebut tidak kurang dari satu.

a) *Mengapa menggunakan maksimum?:* Pemakaian maksimum mempunyai beberapa konsekuensi matematis yang penting. Pertama, karena semua nilai jarak dan waktu tidak negatif, pembagian dengan nilai maksimum menghasilkan rentang $0 \leq D_{norm} \leq 1$ dan $0 \leq T_{norm} \leq 1$. Edge dengan nilai maksimum memperoleh nilai 1, sedangkan nilai lainnya berada di bawahnya.

Kedua, normalisasi maksimum mempertahankan titik nol dan rasio antarnilai. Untuk dua edge e_1 dan e_2 berlaku

$$\frac{D_{norm}(e_1)}{D_{norm}(e_2)} = \frac{D_{eff}(e_1)}{D_{eff}(e_2)}. \quad (13)$$

Artinya, pembagian dengan konstanta yang sama tidak mengubah perbandingan relatif antar-edge. Sifat ini sesuai untuk fungsi biaya yang ingin mempertahankan struktur relatif dari data awal.

Ketiga, normalisasi maksimum tidak menggeser nilai minimum menjadi nol. Hal tersebut berbeda dengan normalisasi min–max,

$$x' = \frac{x - x_{min}}{x_{max} - x_{min}}, \quad (14)$$

yang mengurangi x_{min} dari seluruh data. Pengurangan tersebut dapat mengubah hubungan relatif ketika dua komponen kemudian digabungkan dengan weighted sum. Normalisasi maksimum lebih sederhana karena hanya mengubah skala tanpa mengubah titik asal.

Keempat, nilai 1 mewakili edge dengan nilai terbesar pada atribut tersebut, sehingga bobot J_W dan W_W bekerja pada komponen dengan skala 0–1.

Namun, pilihan ini juga mempunyai konsekuensi. Nilai D_{scale} bergantung pada kendaraan dan bahan bakar karena D_{eff} mengandung biaya bahan bakar, sedangkan T_{scale} bergantung pada jaringan. Oleh karena itu, nilai score absolut dari konfigurasi yang mempunyai kendaraan atau bahan bakar berbeda tidak sebaiknya diperlakukan sebagai ukuran yang sepenuhnya sebanding. Nilai tersebut terutama digunakan untuk menentukan rute dalam konfigurasi yang sama.

E. Fungsi Objektif dan Biaya Kumulatif

Untuk suatu edge, biaya ternormalisasi berbobot dapat ditulis sebagai

$$c(e) = J_W D_{norm}(e) + W_W T_{norm}(e). \quad (15)$$

Tidak terdapat variabel terpisah bernama $c(e)$ pada implementasi. Bentuk weighted sum tersebut diwujudkan langsung di dalam perhitungan cumulative objective cost yang ditampilkan setelah persamaan $g(n)$.

Untuk sebuah jalur P dari start menuju node n , komponen jarak dan waktu merupakan jumlah seluruh edge yang telah dilalui:

$$D_{eff}(P) = \sum_{e \in P} D_{eff}(e), \quad T(P) = \sum_{e \in P} T(e). \quad (16)$$

Akumulasi tiga metrik jalur direalisasikan melalui penjumlahan metrik segmen dengan metrik jalur yang telah diperoleh sebelumnya.

Listing 11. Potongan kode akumulasi jarak, waktu, dan biaya bahan bakar sepanjang jalur.

```
new_metrics = (
    distance + segment_distance,
    total_time + segment_time,
    fuel_cost + segment_fuel
)
```

Dengan demikian biaya yang telah ditempuh oleh A* adalah

$$g(n) = J_W \frac{D_{eff}(P_{s \rightarrow n})}{D_{scale}} + W_W \frac{T(P_{s \rightarrow n})}{T_{scale}}. \quad (17)$$

Perhitungan $g(n)$ pada implementasi dilakukan setelah metrik jalur diperoleh.

Listing 12. Potongan kode perhitungan cumulative objective cost.

```
effective_distance = calculate_effective_distance(
    distance,
    fuel_cost
)

normalized_distance = (
    effective_distance /
    norm["D_SCALE"]
)

normalized_time = (
    time /
    norm["T_SCALE"]
)

return (
    weights["J_W"] * normalized_distance
    +
    weights["W_W"] * normalized_time
)
```

Karena $J_W + W_W = 1$ dan kedua komponen telah dinormalisasi, $g(n)$ merupakan jumlah berbobot yang seimbang secara skala. Bobot kemudian menentukan prioritas: nilai W_W yang lebih besar membuat tambahan waktu memberi pengaruh lebih besar terhadap biaya, sedangkan nilai J_W yang lebih besar meningkatkan pengaruh jarak efektif.

a) Contoh biaya jalur.: Untuk skenario Honda Scoopy–Pertalite dengan layanan Reguler, salah satu jalur yang diperoleh memiliki jarak 19.1 km, waktu 73 menit, dan biaya bahan bakar Rp3,237.29. Jarak efektif totalnya adalah

$$D_{eff}(P) = 19.1 + \frac{3237.29}{1000} = 22.3373 \text{ km-ekuivalen}. \quad (18)$$

Dengan $D_{scale} = 33.6814$, $T_{scale} = 132$, dan bobot $(J_W, W_W) = (0.5, 0.5)$,

$$D_{norm}(P) = \frac{22.3373}{33.6814} = 0.6632, \quad (19)$$

$$T_{norm}(P) = \frac{73}{132} = 0.5530, \quad (20)$$

sehingga

$$g(P) = 0.5(0.6632) + 0.5(0.5530) = 0.6081. \quad (21)$$

Contoh ini memperlihatkan seluruh rantai perhitungan dari jarak fisik hingga nilai objektif akhir.

F. Pembentukan Heuristic dengan Reverse UCS

A* membutuhkan perkiraan biaya dari node n menuju tujuan t . Heuristic yang digunakan tidak didasarkan pada jarak Euclidean, tetapi dihitung dari jaringan jalan. Reverse UCS dijalankan dari node tujuan sehingga cost-to-go untuk seluruh node dapat diperoleh melalui satu proses pencarian untuk masing-masing komponen biaya.

Karena seluruh koneksi jalan bersifat dua arah, membalik arah koneksi tidak mengubah himpunan koneksi. Makna “reverse” dalam konteks ini adalah titik awal pencarian diletakkan pada tujuan. Hasilnya adalah tabel biaya minimum dari setiap node menuju tujuan.

Dua nilai cost-to-go dihitung secara terpisah:

$$D_{LB}(n) = \min_{P:n \rightarrow t} \sum_{e \in P} D_{eff}(e), \quad (22)$$

Untuk komponen jarak efektif, fungsi Reverse UCS dipanggil dengan biaya edge yang dibentuk dari jarak dan biaya bahan bakar.

Listing 13. Potongan kode pembentukan lower bound jarak efektif.

```
distance_lb = reverse_ucs_lower_bounds(
    goal,
    lambda u, v, _time: calculate_effective_distance(
        DISTANCE[u][v],
        DISTANCE[u][v] * fuel_price_km
    )
)
```

$$T_{LB}(n) = \min_{P:n \rightarrow t} \sum_{e \in P} T(e). \quad (23)$$

Untuk komponen waktu, biaya edge yang diberikan kepada Reverse UCS adalah waktu tempuh yang tersimpan pada graf.

Listing 14. Potongan kode pembentukan lower bound waktu.

```
time_lb = reverse_ucs_lower_bounds(
    goal,
    lambda _u, _v, travel_time: travel_time
)
```

Keduanya kemudian dinormalisasi dan digabungkan:

$$h(n) = J_W \frac{D_{LB}(n)}{D_{scale}} + W_W \frac{T_{LB}(n)}{T_{scale}}. \quad (24)$$

Penggabungan kedua lower bound dilakukan pada fungsi heuristic berikut.

Listing 15. Potongan kode pembentukan heuristic dari dua lower bound.

```
def heuristic(node):
    return (
        weights["J_W"]
        * (distance_lb[node] / norm["D_SCALE"])
        +
        weights["W_W"]
        * (time_lb[node] / norm["T_SCALE"])
    )
```

a) Mengapa heuristic tersebut dapat digunakan?: Untuk setiap jalur P dari n menuju t , berlaku

$$D_{LB}(n) \leq D_{eff}(P) \quad (25)$$

dan

$$T_{LB}(n) \leq T(P). \quad (26)$$

Karena J_W , W_W , D_{scale} , dan T_{scale} bernilai positif atau nol sesuai definisinya, kedua ketaksamaan tersebut dapat dikalikan dengan bobot dan dijumlahkan. Akibatnya,

$$h(n) \leq C(P) \quad (27)$$

untuk setiap jalur P , termasuk jalur optimal. Jadi $h(n)$ merupakan lower bound atau heuristic admissible terhadap biaya menuju tujuan.

Hal penting lainnya adalah bahwa $h(n)$ tidak harus sama dengan biaya optimal sebenarnya. Jalur yang memberikan jarak efektif minimum dapat berbeda dari jalur yang memberikan waktu minimum. Penjumlahan kedua minimum tersebut cenderung menghasilkan nilai yang lebih kecil atau sama dengan minimum dari jumlah keduanya. Justru sifat sebagai batas bawah inilah yang diperlukan oleh A*: heuristic membantu mengarahkan pencarian tanpa melebihi biaya yang sebenarnya masih harus ditempuh.

G. Fungsi Evaluasi A*

Setelah $g(n)$ dan $h(n)$ tersedia, A* menggunakan

$$f(n) = g(n) + h(n). \quad (28)$$

Nilai f untuk kandidat tetangga dihitung secara langsung setelah g baru dan heuristic tersedia.

Listing 16. Potongan implementasi pembaruan nilai g , h , dan f pada A*.

```
new_g = calculate_g_score(
    new_metrics[0],
    new_metrics[1],
    new_metrics[2],
    weights,
    norm
)

improved = (
    new_g < g_score[neighbor]
```

```
)

h_child = heuristic(neighbor)
f_child = new_g + h_child

if improved:
    came_from[neighbor] = current
    g_score[neighbor] = new_g
    metrics[neighbor] = new_metrics

    heapq.heappush(
        queue,
        (
            f_child,
            next(tie_breaker),
            neighbor
        )
    )
```

Nilai $g(n)$ menyatakan biaya yang telah diketahui dari start menuju n , sedangkan $h(n)$ memperkirakan biaya minimum yang masih diperlukan dari n menuju tujuan. Node dengan nilai $f(n)$ paling kecil diprioritaskan untuk dikembangkan.

Dengan heuristic yang admissible dan bobot biaya nonnegatif, nilai $f(n)$ memberikan batas bawah terhadap biaya solusi yang melewati node tersebut. Karena heuristic yang digunakan juga dibentuk dari shortest path untuk komponen-komponen biaya, pencarian dapat diarahkan ke node yang mempunyai prospek biaya total lebih rendah tanpa mengorbankan optimalitas terhadap fungsi objektif yang telah didefinisikan.

V. ALGORITMA DAN PSEUDOCODE

Pseudocode berikut menjelaskan dua tahap utama. Tahap pertama menghitung cost-to-go dari tujuan menuju seluruh node. Tahap kedua menggunakan cost-to-go tersebut sebagai heuristic untuk mencari jalur dari start menuju tujuan.

A. Reverse UCS

Reverse UCS dimulai dari tujuan dengan biaya nol. Setiap kali node dengan biaya terkecil dikeluarkan dari priority queue, tetangga yang terhubung diperiksa. Jika jalur baru memberikan biaya lebih kecil daripada nilai yang telah diketahui, nilai tersebut diperbarui dan dimasukkan kembali ke priority queue. Prosedur yang sama digunakan untuk lower bound jarak efektif dan waktu.

Algoritma 1. Reverse UCS untuk Cost-to-Go

Require: Graf $G = (V, E)$, tujuan t , biaya edge C

Ensure: $L(n)$ untuk seluruh $n \in V$

```
1: for all  $n \in V$  do
2:    $L(n) \leftarrow \infty$ 
3: end for
4:  $L(t) \leftarrow 0$ 
5: Masukkan  $(0, t)$  ke priority queue  $Q$ 
6: while  $Q$  tidak kosong do
7:    $(c, u) \leftarrow$  elemen dengan biaya terkecil dari  $Q$ 
8:   if  $c > L(u)$  then
9:     Lanjutkan ke iterasi berikutnya
10:  end if
11:  for all  $v$  yang terhubung dengan  $u$  do
12:     $c' \leftarrow L(u) + C(v, u)$ 
```

```

13:         if  $c' < L(v)$  then
14:              $L(v) \leftarrow c'$ 
15:             Masukkan  $(L(v), v)$  ke  $Q$ 
16:         end if
17:     end for
18: end while
19: return  $L$ 

```

Listing 17. Implementasi Reverse UCS untuk menghitung cost-to-go.

```

distances = {
    node: float("inf")
    for node in GRAPH
}

distances[goal] = 0.0

queue = [(0.0, goal)]

while queue:
    current_cost, current = heapq.heappop(queue)

    if current_cost > distances[current]:
        continue

    for neighbor, edge_cost in reverse_graph[current].items():
        candidate = current_cost + edge_cost

        if candidate < distances[neighbor]:
            distances[neighbor] = candidate
            heapq.heappush(
                queue,
                (candidate, neighbor)
            )

return distances

```

Cara kerja Reverse UCS. Pertama, seluruh node diberi nilai cost-to-go tak hingga, kemudian node tujuan diberi nilai nol karena tidak diperlukan biaya tambahan untuk mencapai tujuan dari dirinya sendiri. Selanjutnya, pasangan $(0, t)$ dimasukkan ke priority queue. Node dengan biaya terkecil selalu diambil lebih dahulu. Untuk setiap tetangga, algoritma membentuk kandidat biaya baru dengan menjumlahkan biaya yang telah diketahui dengan biaya edge menuju tetangga tersebut. Jika kandidat lebih kecil daripada nilai yang tersimpan, nilai diperbarui dan pasangan baru dimasukkan kembali ke queue. Pemeriksaan $\text{current_cost} > \text{distances}[\text{current}]$ berfungsi mengabaikan entri lama pada queue yang sudah tidak memberikan nilai terbaik. Proses berulang sampai queue kosong sehingga diperoleh cost-to-go untuk seluruh node.

Implementasi mempertahankan urutan operasi dari kode sumber: inisialisasi cost-to-go, penetapan tujuan, pengambilan kandidat dari heap, pemeriksaan kandidat lama, pembentukan biaya baru, pembaruan nilai, dan pengembalian dictionary jarak.

B. A*

Setelah heuristic tersedia, A* memulai pencarian dari node awal. Nilai g menyimpan biaya dari start, sedangkan $f = g + h$ digunakan untuk menentukan node yang diprioritaskan. Ketika sebuah tetangga menghasilkan g' yang lebih kecil, nilai tersebut menggantikan nilai sebelumnya dan parent diperbarui untuk merekonstruksi jalur.

Pseudocode berikut merangkum proses inisialisasi, pemilihan node berdasarkan nilai f , evaluasi tetangga, dan pembaruan parent. Proses tersebut diimplementasikan pada pencarian A* yang digunakan dalam eksperimen.

Algoritma 2. A* Search dengan Heuristic Reverse UCS

Require: Graf G , start s , tujuan t , heuristic h , biaya edge C

Ensure: Jalur P atau tidak ada solusi

```

1: for all  $n \in V$  do
2:      $g(n) \leftarrow \infty$ 
3:      $\text{parent}(n) \leftarrow \text{NULL}$ 
4: end for
5:  $g(s) \leftarrow 0$ 
6: Masukkan  $(h(s), s)$  ke priority queue  $Q$ 
7: while  $Q$  tidak kosong do
8:     Ambil node  $u$  dengan  $f(u) = g(u) + h(u)$  terkecil
9:     if  $u = t$  then
10:         Rekonstruksi jalur melalui  $\text{parent}$ 
11:         return  $P$ 
12:     end if
13:     for all  $v$  tetangga dari  $u$  do
14:          $g' \leftarrow g(u) + C(u, v)$ 
15:         if  $g' < g(v)$  then
16:              $g(v) \leftarrow g'$ 
17:              $\text{parent}(v) \leftarrow u$ 
18:              $f(v) \leftarrow g(v) + h(v)$ 
19:             Masukkan  $(f(v), v)$  ke  $Q$ 
20:         end if
21:     end for
22: end while
23: return tidak ada solusi

```

Cara kerja A*. Algoritma dimulai dengan memberikan $g(s) = 0$ pada node awal dan memasukkan node tersebut ke priority queue dengan prioritas $f(s) = g(s) + h(s)$. Pada setiap iterasi, node dengan nilai f terkecil dipilih untuk dikembangkan. Jika node yang dipilih adalah tujuan, parent digunakan untuk menelusuri kembali jalur dari tujuan ke start. Jika belum mencapai tujuan, setiap tetangga dievaluasi dengan menghitung kandidat biaya baru g' . Apabila g' lebih kecil daripada g yang tersimpan untuk tetangga tersebut, parent dan nilai g diperbarui, kemudian nilai $f = g + h$ dihitung menggunakan heuristic Reverse UCS dan node dimasukkan ke priority queue. Dengan mekanisme ini, g mewakili biaya yang sudah ditempuh, sedangkan h memberikan batas bawah biaya yang masih diperlukan menuju tujuan.

Implementasi pembaruan g , h , dan f menggunakan nilai biaya kumulatif dan heuristic pada setiap tetangga yang dievaluasi.

Implementasi A* menggunakan pembaruan nilai g , heuristic h , dan f seperti pada kode sumber di atas. Dengan heuristic admissible dan biaya edge nonnegatif, pencarian diarahkan berdasarkan estimasi biaya total tanpa melebihi biaya yang masih diperlukan menuju tujuan.

VI. SKENARIO

Empat kelompok kendaraan–bahan bakar digunakan, dan setiap kelompok dipadukan dengan empat layanan. Dengan

demikian terdapat

$$4 \times 4 = 16 \quad (29)$$

skenario. Pada setiap kelompok, perubahan layanan hanya mengubah bobot (J_W, W_W) sehingga pengaruh perubahan rute dapat diamati pada fungsi objektif yang sama.

TABLE IV
KONFIGURASI KENDARAAN, BAHAN BAKAR, DAN LAYANAN PADA 16
SKENARIO

No.	Kendaraan	BBM	Layanan
1	V1	B1	S1
2	V1	B1	S2
3	V1	B1	S3
4	V1	B1	S4
5	V2	B1	S1
6	V2	B1	S2
7	V2	B1	S3
8	V2	B1	S4
9	V3	B2	S1
10	V3	B2	S2
11	V3	B2	S3
12	V3	B2	S4
13	V4	B3	S1
14	V4	B3	S2
15	V4	B3	S3
16	V4	B3	S4

Tabel IV menggunakan kode yang sama dengan tabel kendaraan, bahan bakar, dan layanan pada bagian informasi data. Dengan demikian, V1–V4 mengacu pada kendaraan, B1–B3 pada jenis bahan bakar, dan S1–S4 pada paket layanan. Penggunaan kode menghindari pengulangan nama yang panjang pada tabel hasil.

VII. HASIL DAN PEMBAHASAN

Hasil pencarian untuk 16 skenario meliputi rute, jarak, waktu tempuh, biaya bahan bakar, dan nilai fungsi objektif g . Seluruh hasil tersebut disajikan pada Tabel V.

TABLE V
HASIL PENCARIAN PADA 16 SKENARIO

No.	Rute	Jarak (km)	Waktu (min)	Biaya (Rp)	g
1	B → AE → AD → AW → P → AF → N	19.1	73	3,237.29	0.608112
2	B → Y → AF → N	31.7	20	5,372.88	0.341351
3	B → Y → AF → N	31.7	20	5,372.88	0.246433
4	B → W → AE → AD → AW → P → AF → N	13.1	105	2,220.34	0.522980
5	A → AA → AL → AM → AV → B	13.4	41	2,211.22	0.387942
6	A → AC → AB → AD → AE → B	15.7	38	2,590.76	0.339331
7	A → AC → AB → AD → AE → B	15.7	38	2,590.76	0.313605
8	A → AC → AB → AD → AE → W → B	9.7	70	1,600.66	0.375505
9	E → AQ → AG → AF → P	18.3	51	5,267.59	0.510890
10	E → AQ → Y → AF → P	19.0	50	5,469.08	0.434975
11	E → AQ → Y → AF → P	19.0	50	5,469.08	0.406881
12	E → AQ → AG → P	17.8	58	5,123.67	0.582323
13	K → AL → AA → A → AC → AB → AD → AW → P	15.3	70	5,641.88	0.530777
14	K → AL → AA → A → AC → AB → AR → P	23.1	57	8,518.12	0.505871
15	K → AL → AA → AK → A → AC → AB → AR → P	34.2	51	12,611.25	0.466477
16	K → AS → AX → AD → AW → P	8.0	107	2,950.00	0.384343

Hasil pada Tabel V menunjukkan adanya perubahan rute sesuai bobot layanan. Pada Honda Scoopy–Pertalite, layanan Kilat dan Express menggunakan rute yang sama dengan waktu 20 menit, sedangkan layanan Hemat menghasilkan rute 13.1 km dengan waktu 105 menit. Pada Honda Beat–Pertalite, layanan Hemat menghasilkan rute 9.7 km dengan waktu 70 menit, sementara layanan Kilat dan Express menggunakan rute 15.7 km dengan waktu 38 menit. Pada Honda Vario 160–Pertamax, layanan Kilat dan Express menghasilkan rute 19.0 km dengan waktu 50 menit. Pada Yamaha NMAX–Pertamax Turbo, layanan Express menghasilkan rute 34.2 km dengan waktu 51 menit, sedangkan layanan Hemat menghasilkan rute 8.0 km dengan waktu 107 menit.

Nilai g merupakan gabungan jarak efektif dan waktu yang telah dinormalisasi sesuai bobot layanan. Nilai tersebut berbeda pada setiap skenario, misalnya 0.246433 pada skenario Express Honda Scoopy–Pertalite dan 0.384343 pada skenario Hemat Yamaha NMAX–Pertamax Turbo. Nilai g antarkonfigurasi tidak digunakan sebagai peringkat absolut karena D_{scale} dipengaruhi oleh kendaraan dan bahan bakar melalui jarak efektif. Secara keseluruhan, hasil menunjukkan bahwa perubahan bobot layanan dapat mengubah keseimbangan antara jarak, waktu, dan biaya bahan bakar.

```

EKSPERIMEN 01 / 16
Grup 1 dari 4 | B -> N | Honda Scoopy + Peralite (RON 90) | Rp169.49/km

Rute Optimal      : B -> AE -> AD -> AW -> P -> AF -> N
Kendaraan Digunakan : Honda Scoopy (59.0 km/L)
Jenis BBM         : Peralite (RON 90)
Paket Layanan     : Regular (Seimbang)
Bobot Jarak (J_M)  : 0.50
Bobot Waktu (W_M)  : 0.50
Total Jarak       : 19.10 km
Total Waktu       : 73.0 menit
Estimasi Biaya Bensin : Rp 3237.29

```

Fig. 2. Screenshot implementasi skenario 1.

```

EKSPERIMEN 02 / 16
Grup 1 dari 4 | B -> N | Honda Scoopy + Peralite (RON 90) | Rp169.49/km

Rute Optimal      : B -> Y -> AF -> N
Kendaraan Digunakan : Honda Scoopy (59.0 km/L)
Jenis BBM         : Peralite (RON 90)
Paket Layanan     : Kilat (Prioritas Waktu)
Bobot Jarak (J_M)  : 0.20
Bobot Waktu (W_M)  : 0.80
Total Jarak       : 31.70 km
Total Waktu       : 20.0 menit
Estimasi Biaya Bensin : Rp 5372.88

```

Fig. 3. Screenshot implementasi skenario 2.

```

EKSPERIMEN 03 / 16
Grup 1 dari 4 | B -> N | Honda Scoopy + Peralite (RON 90) | Rp169.49/km

Rute Optimal      : B -> Y -> AF -> N
Kendaraan Digunakan : Honda Scoopy (59.0 km/L)
Jenis BBM         : Peralite (RON 90)
Paket Layanan     : Express (Waktu Agresif)
Bobot Jarak (J_M)  : 0.10
Bobot Waktu (W_M)  : 0.90
Total Jarak       : 31.70 km
Total Waktu       : 20.0 menit
Estimasi Biaya Bensin : Rp 5372.88

```

Fig. 4. Screenshot implementasi skenario 3.

```

EKSPERIMEN 04 / 16
Grup 1 dari 4 | B -> N | Honda Scoopy + Peralite (RON 90) | Rp169.49/km

Rute Optimal      : B -> W -> AE -> AD -> AW -> P -> AF -> N
Kendaraan Digunakan : Honda Scoopy (59.0 km/L)
Jenis BBM         : Peralite (RON 90)
Paket Layanan     : Hemat (Prioritas Jarak)
Bobot Jarak (J_M)  : 0.80
Bobot Waktu (W_M)  : 0.20
Total Jarak       : 13.10 km
Total Waktu       : 195.0 menit
Estimasi Biaya Bensin : Rp 2220.34

```

Fig. 5. Screenshot implementasi skenario 4.

VIII. KESIMPULAN

Implementasi A* dengan heuristic Reverse UCS menghasilkan rute untuk seluruh 16 skenario berdasarkan fungsi objektif yang menggabungkan jarak efektif dan waktu tempuh. Heuristic yang dihitung melalui Reverse UCS memberikan cost-to-go berdasarkan jaringan yang digunakan dalam pencarian, sedangkan integrasi biaya bahan bakar memungkinkan karakteristik kendaraan dan jenis bahan bakar ikut memengaruhi jarak efektif. Dalam konteks penggunaan nyata, pendekatan ini dapat digunakan ketika tujuan pengiriman perlu mempertimbangkan lebih dari satu aspek, misalnya waktu tempuh

sekali konsumsi biaya bahan bakar. Bobot layanan juga memungkinkan fungsi objektif disesuaikan dengan karakteristik layanan pengiriman.

Keterbatasannya adalah adanya komputasi tambahan untuk membentuk heuristic sebelum A* dijalankan, sehingga terdapat overhead pemrosesan dibandingkan pencarian yang tidak memerlukan tahap tersebut. Hasil juga bergantung pada bobot layanan dan konstanta normalisasi yang ditentukan dalam model. Pada implementasi ini, waktu tempuh bersifat statis dan efisiensi bahan bakar menggunakan nilai nominal kendaraan, sehingga kondisi nyata seperti kemacetan, perubahan kecepatan, kondisi jalan, cuaca, beban kendaraan, dan perubahan harga bahan bakar belum direpresentasikan secara dinamis. Selain itu, model belum menerapkan batasan operasional seperti kapasitas kendaraan, batas waktu pengantaran, biaya maksimum, atau perubahan kondisi jaringan secara real-time.

Dengan demikian, pendekatan yang digunakan sesuai untuk menunjukkan penerapan A* pada masalah rute multi-kriteria dengan data jaringan dan parameter yang telah ditentukan. Untuk penggunaan operasional di dunia nyata, model perlu dilengkapi dengan data perjalanan aktual, pembaruan kondisi jaringan, parameter kendaraan yang lebih realistis, serta batasan operasional yang relevan.